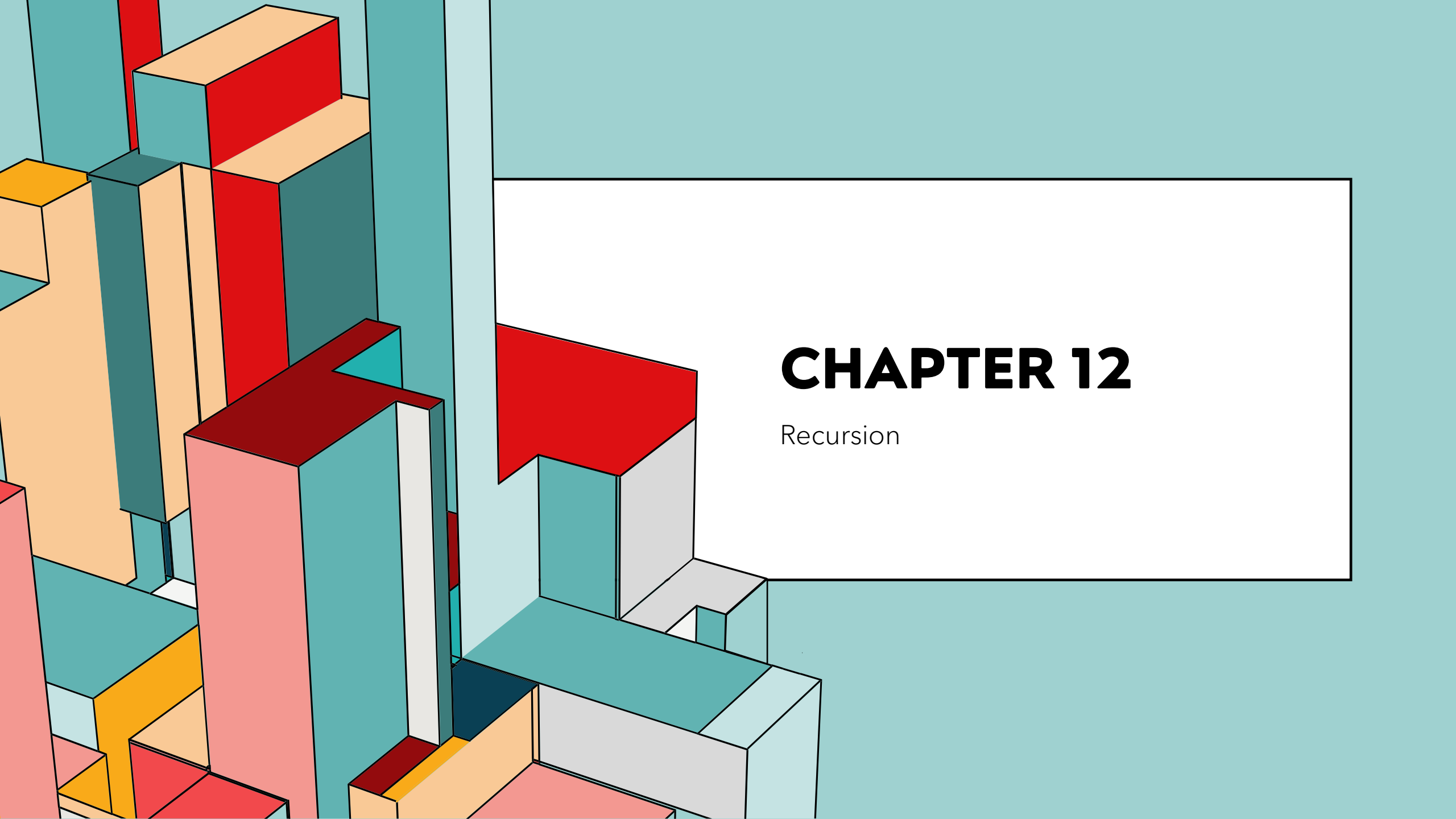


The background features a collection of 3D rectangular blocks in various colors including teal, orange, red, and pink, arranged in a complex, overlapping structure on the left side. A large white rectangular box with a thin black border occupies the right half of the image, containing the chapter title and subtitle.

# CHAPTER 12

Recursion

# จุดประสงค์

- เข้าใจความหมายและการทำงานของโปรแกรม Recursion ได้
- สามารถพัฒนาโปรแกรม Recursion อย่างง่ายได้

# ทดสอบก่อนเรียน

- $\text{test}(2) = ?$

algorithm  $\text{test}(n)$

int  $\text{res}$

if ( $n == -3$ )

$\text{res} = 5$

else

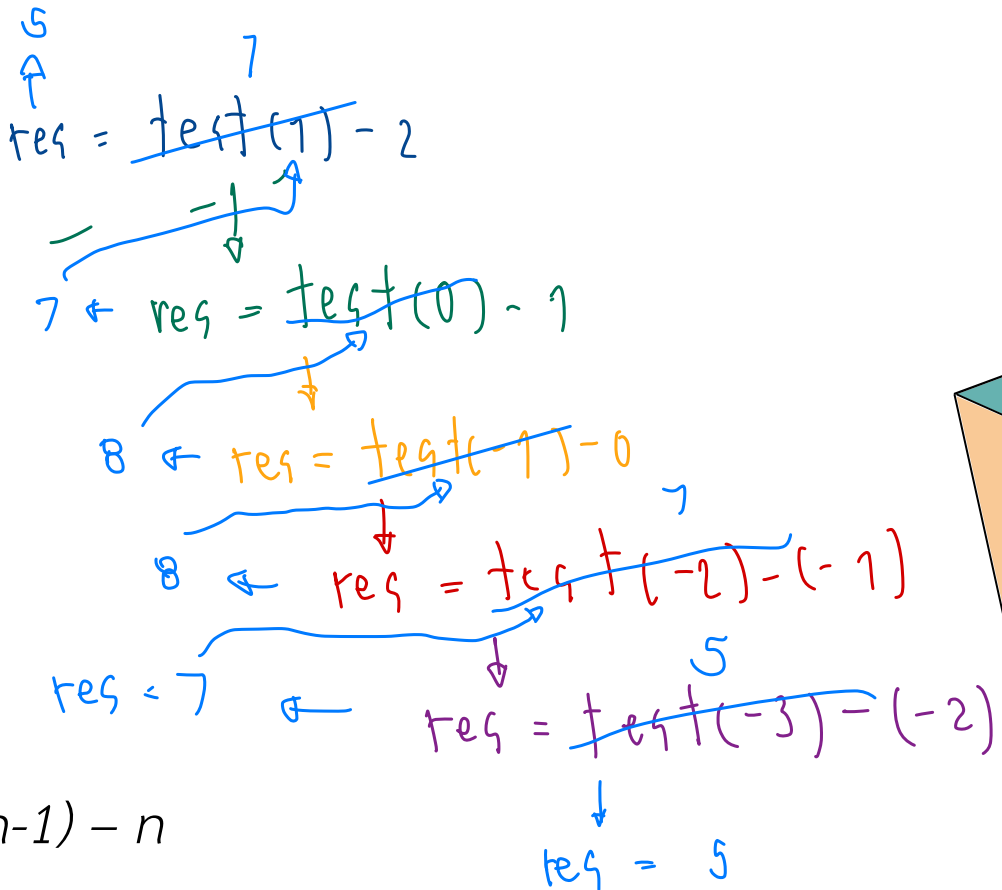
$\text{res} = \text{test}(n-1) - n$

end if

return ( $\text{res}$ )

end test

∴  $\text{test}(2) = 5$



# อัลกอริทึมที่มีการทำซ้ำ (REPETITIVE ALGORITHM)

❖ การเขียนอัลกอริทึมที่มีการเรียกซ้ำทำได้ 2 รูปแบบ

- Iteration : โปรแกรมที่มีการทำงานซ้ำตามเงื่อนไขที่กำหนด เช่น การใช้ for-loop, while-loop

- Ex.

```
int i = 1;
while (i<5) {
    printf("Number : %d", i);
    i++;
}
```

- Recursion : โปรแกรมย่อยที่มีการเรียกใช้ตัวเอง

# CASE STUDY : FACTORIAL

$$\text{Factorial}(n) = \begin{cases} 1 & , n=0 \\ n*(n-1)*(n-2)*...*3*2*1 & , n>0 \end{cases}$$

$$\text{Factorial}(4) = 4*3*2*1 = 24$$

# FACTORIAL : ITERATIVE SOLUTION

$$n = 4$$

$$4! = 4 \times 3 \times 2 \times 1 = 1 \times 2 \times 3 \times 4$$

**Algorithm** iterativeFactorial (n)  $n!$

Calculates the factorial of a number using a loop.

Pre n is the number to be raised factorially

Post  $n!$  is returned

1 set  $i$  to 1  $i = 1$

2 set factN to 1

3 loop ( $i \leq n$ )

1 set factN to factN \* i

2 increment i

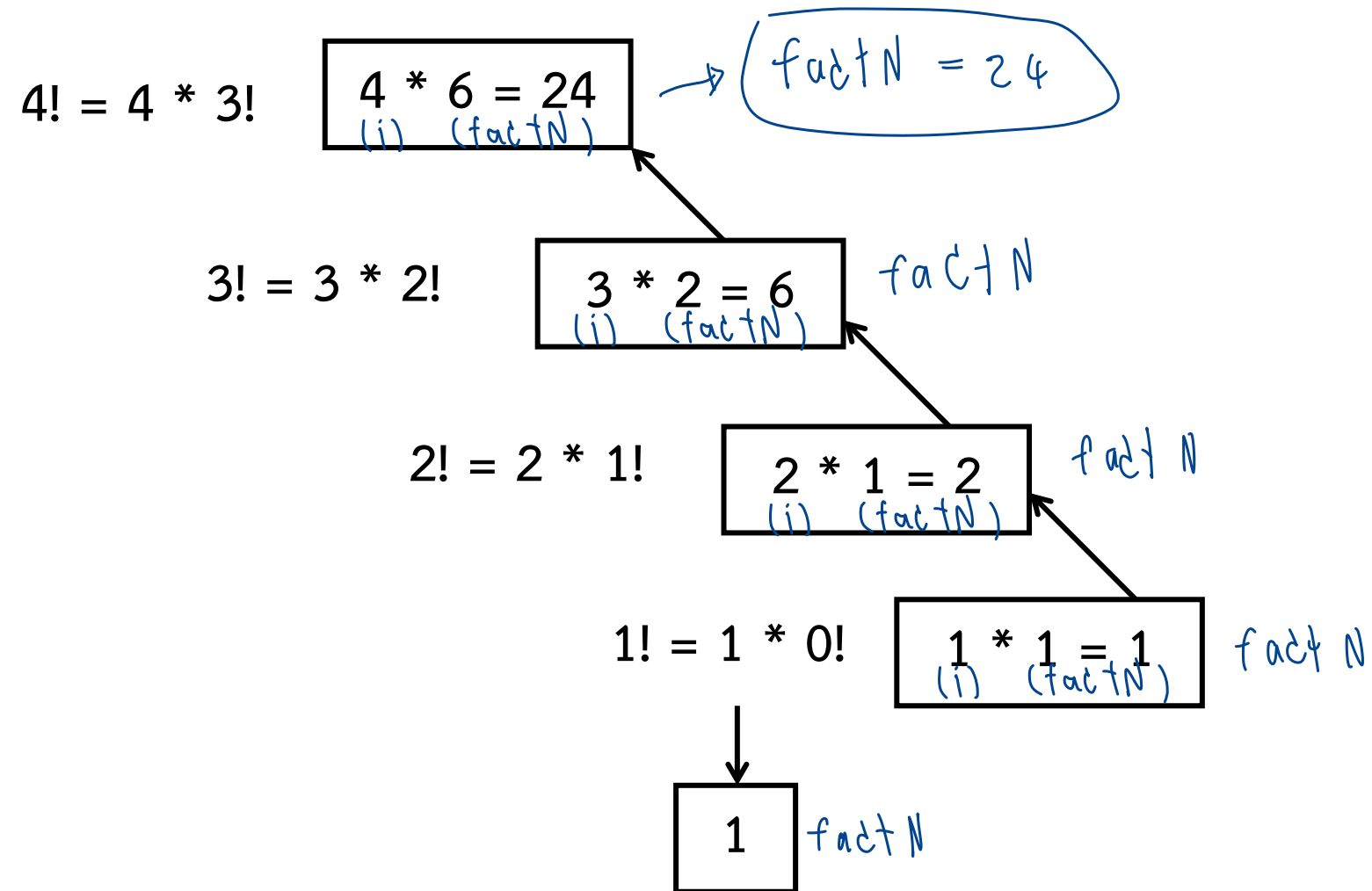
4 end loop

5 return factN

**end** iterativeFactorial

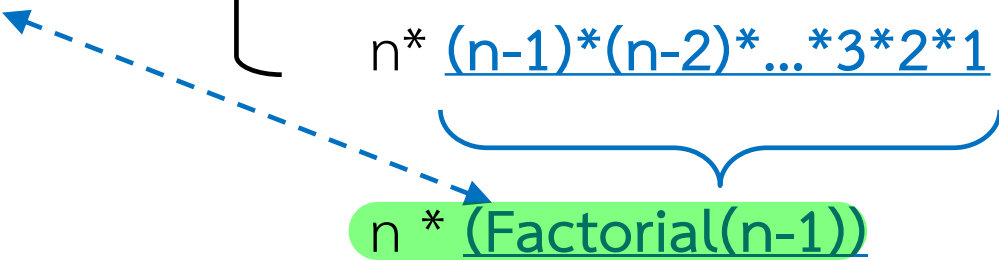
$$\begin{aligned} \text{factN} &= \text{factN} * i \\ &= \text{factN} * 1 \\ &= 1 \\ \text{factN} &= \text{factN} + 2 \end{aligned}$$

# CASE STUDY : FACTORIAL(4)



# FACTORIAL : RECURSION DEFINED

$$\text{Factorial}(n) = \begin{cases} 1 & , n=0 \\ n * \underbrace{(n-1)*(n-2)*\dots*3*2*1}_{\text{Factorial}(n-1)} & , n>0 \end{cases}$$





# FACTORIAL : RECURSIVE SOLUTION

```
Algorithm recursiveFactorial (n)
Calculates factorial of a number using recursion.
  Pre    n  is the number being raised factorially
  Post   n! is returned
1  if (n equals 0)
  1  return 1
2  else
  1  return (n * recursiveFactorial (n - 1))
3  end if
end recursiveFactorial
```

```
program factorial
1 factN = recursiveFactorial(3)
2 print (factN)
end factorial
```

3!

FACTORIAL :

CALLING A RECURSIVE  
ALGORITHM

**Algorithm** recursiveFactorial (n)

1 if (n equals 0)

1 return 1

2 else

1 return (n x recursiveFactorial (n - 1))

3 end if

**end** recursiveFactorial

**Algorithm** recursiveFactorial (n)

1 if (n equals 0)

1 return 1

2 else

1 return (n x recursiveFactorial (n - 1))

3 end if

**end** recursiveFactorial

**Algorithm** recursiveFactorial (n)

1 if (n equals 0)

1 return 1

2 else

1 return (n x recursiveFactorial (n - 1))

3 end if

**end** recursiveFactorial

**Algorithm** recursiveFactorial (n)

1 if (n equals 0)

1 return 1

2 else

1 return (n x recursiveFactorial (n - 1))

3 end if

**end** recursiveFactorial

# THE DESIGN METHODOLOGY

- การเรียกตัวเองซ้ำ จะประกอบด้วยการทำงาน 2 ส่วน
  - ส่วนที่ทำการหยุดการเรียกซ้ำ -> *base case*
  - ส่วนที่ทำการเรียกซ้ำ -> *general case*

- Factorial

- *Base case* :  $\text{factorial}(0)$
  - *General case* :  $n * \text{factorial}(n-1)$
- ทำในขั้น base case

# RULES FOR DESIGNING

- ❖ การเรียกตัวเองซ้ำ จะเรียกตัวเองไปเรื่อยๆ เพื่อลดขนาดของปัญหาลงจนมีการหยุดการเรียกซ้ำ
- ❖ การเขียนโปรแกรมย่อยที่มีการเรียกตัวเองซ้ำ จะต้อง
  - กำหนดส่วนที่ทำให้หยุดการเรียกซ้ำ
  - กำหนดส่วนที่ทำการเรียกซ้ำ

# LIMITATIONS OF RECURSION

❖ เราไม่ควรใช้ลักษณะการเรียกตัวเองซ้ำ เมื่อ

- อัลกอริทึมหรือโครงสร้างข้อมูลที่ใช้ ไม่เหมาะกับการเรียกตัวเองซ้ำ
- การเรียกตัวเองซ้ำ ทำให้ยุ่งยากมากขึ้น (โค้ดยาวขึ้น เข้าใจยาก)
- ใช้เนื้อที่และเวลาในการทำงานมาก

# PRINT REVERSE EXAMPLE

```
Algorithm printReverse (data)
Print keyboard data in reverse.
    Pre  nothing
    Post data printed in reverse
1  if (end of input)
    1  return
2  end if
3  read data
4  printReverse (data)
Have reached end of input: print nodes
5  print data
6  return
end printReverse
```

# PRINT REVERSE EXAMPLE (CONT.)

- ❖ พิจารณาว่าการทำงานนี้ควรใช้การเรียกตัวเองซ้ำหรือไม่
  - อัลกอริทึมนี้ไม่เหมาะกับการเรียกตัวเองซ้ำ เนื่องจากไม่ใช่ logarithmic algorithm
  - อัลกอริทึมนี้เข้าใจได้ยาก
  - ประสิทธิภาพการทำงานของอัลกอริทึมคิดเป็น  $O(n)$
- ❖ **การทำงานนี้ไม่ควรใช้การเรียกตัวเองซ้ำ**

# RECURSIVE EXAMPLE

- ❖ Greatest Common Divisor
- ❖ Fibonacci number
- ❖ Towers of Hanoi



# GREATEST COMMON DIVISOR

$$\text{gcd}(a, b) = \begin{cases} a & \text{if } b = 0 \\ b & \text{if } a = 0 \\ \text{gcd}(b, a \bmod b) & \text{otherwise} \end{cases}$$

# GREATEST COMMON DIVISOR (CONT.)

**Algorithm gcd (a, b)**

Calculates greatest common divisor using the Euclidean algorithm.

Pre a and b are positive integers greater than 0

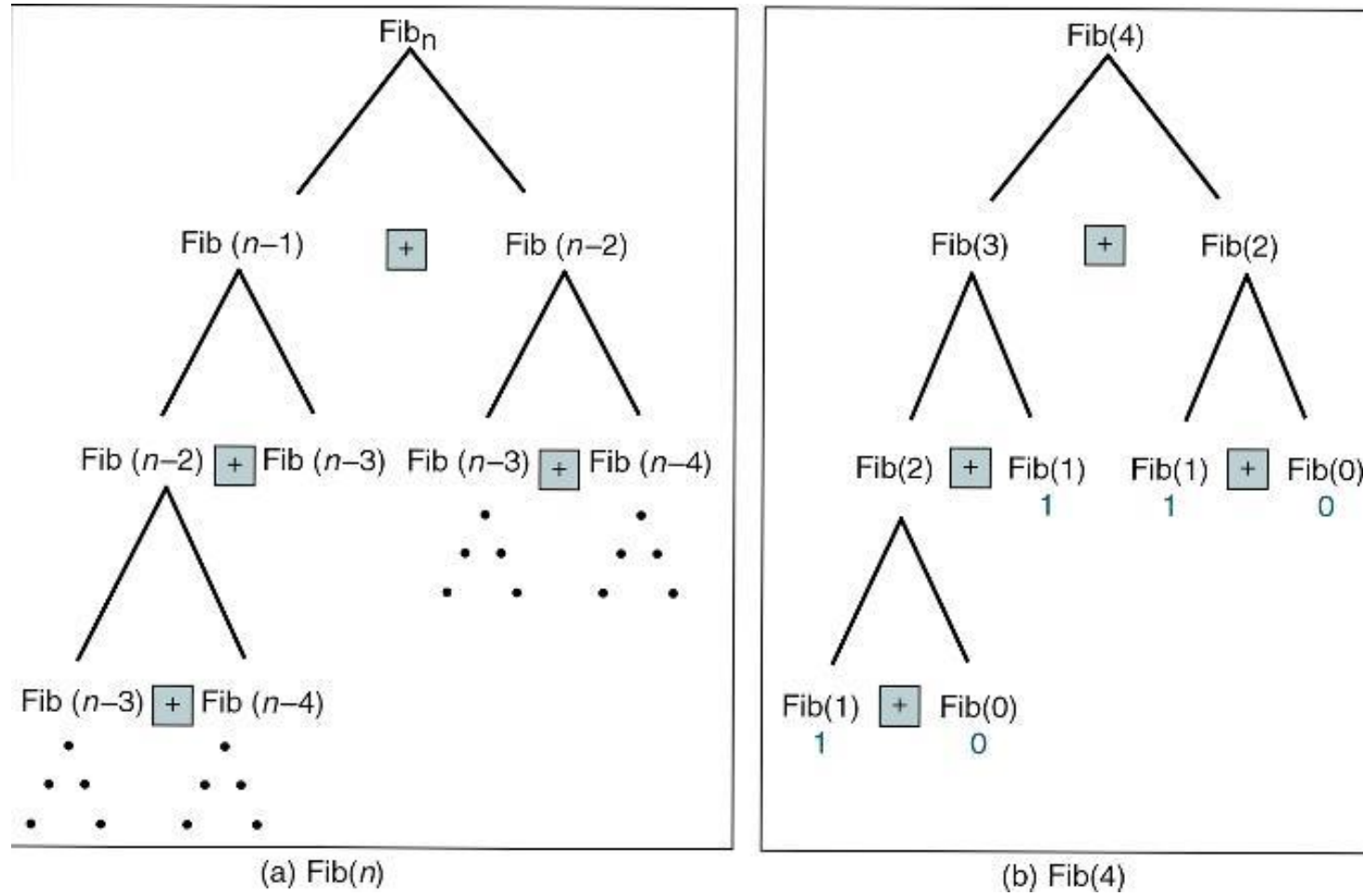
Post greatest common divisor returned

```
1 if (b equals 0)
  1 return a
2 end if
3 if (a equals 0)
  2 return b
4 end if
5 return gcd (b, a mod b)
end gcd
```

# FIBONACCI NUMBERS

$$\text{Fibonacci}(n) = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ \text{Fibonacci}(n - 1) + \text{Fibonacci}(n - 2) & \text{otherwise} \end{cases}$$

# FIBONACCI NUMBERS (CONT.)

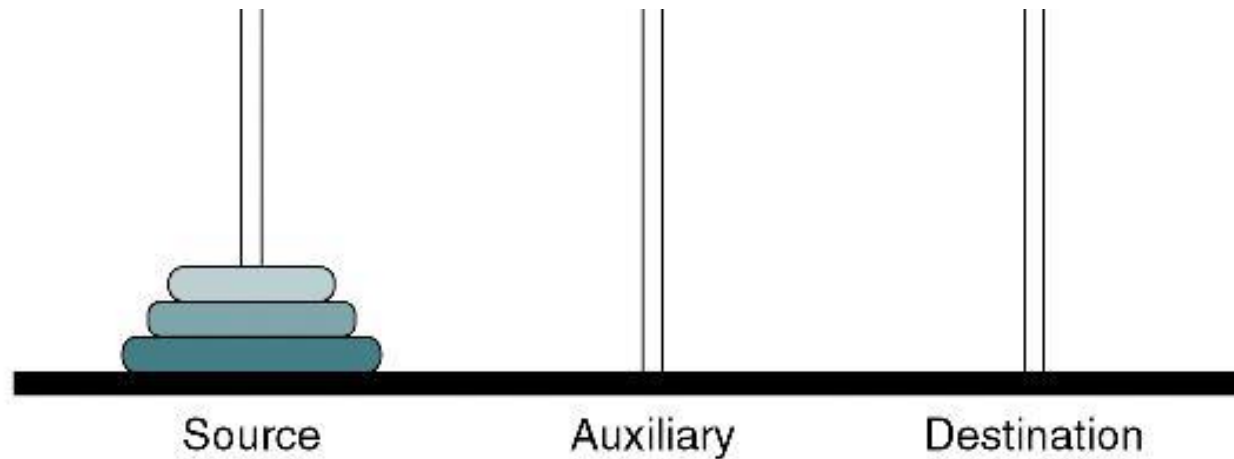


# FIBONACCI NUMBERS (CONT.)

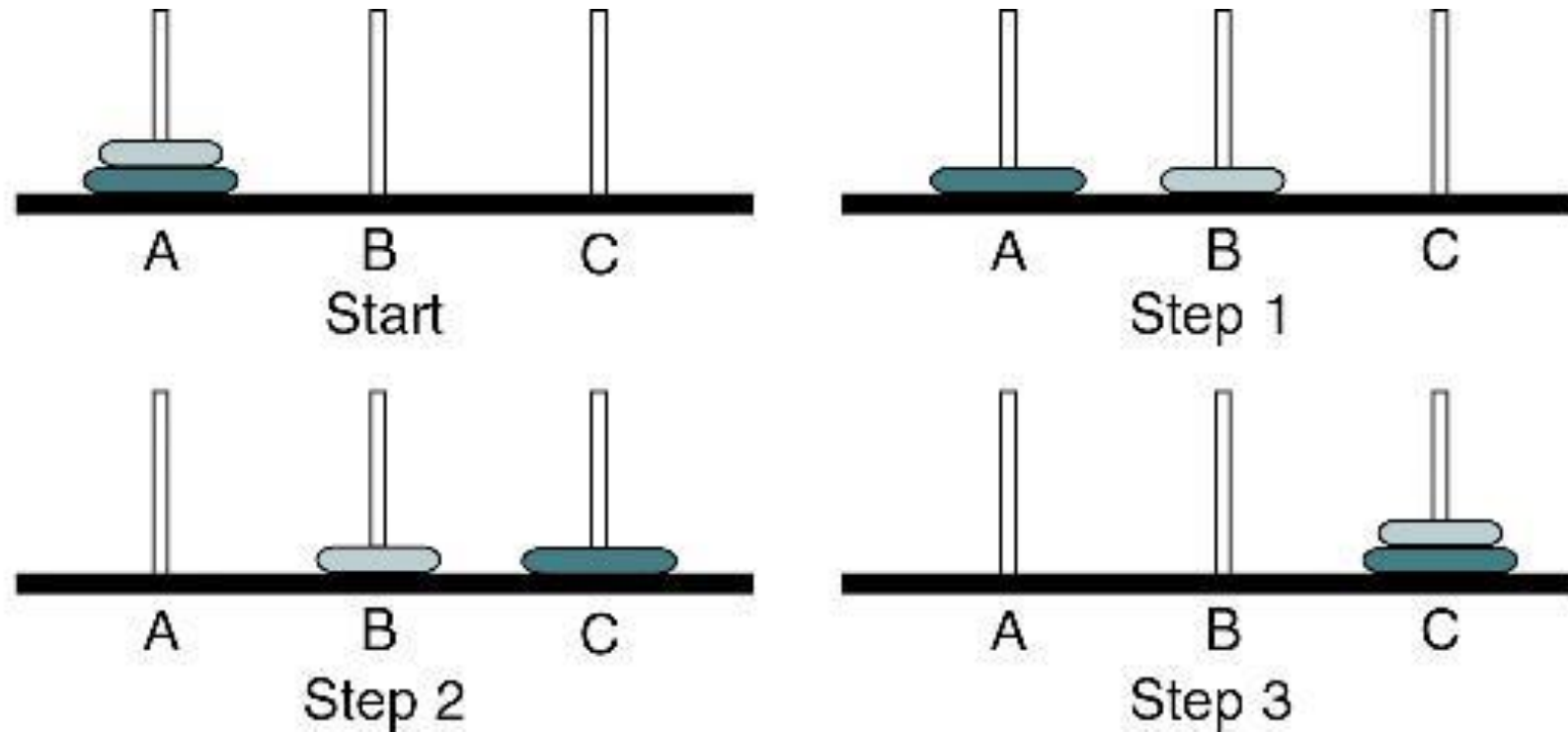
| <b>fib(<i>n</i>)</b> | <b>Calls</b> | <b>fib(<i>n</i>)</b> | <b>Calls</b> |
|----------------------|--------------|----------------------|--------------|
| 1                    | 1            | 11                   | 287          |
| 2                    | 3            | 12                   | 465          |
| 3                    | 5            | 13                   | 753          |
| 4                    | 9            | 14                   | 1219         |
| 5                    | 15           | 15                   | 1973         |
| 6                    | 25           | 20                   | 21,891       |
| 7                    | 41           | 25                   | 242,785      |
| 8                    | 67           | 30                   | 2,692,573    |
| 9                    | 109          | 35                   | 29,860,703   |
| 10                   | 177          | 40                   | 331,160,281  |

# THE TOWERS OF HANOI

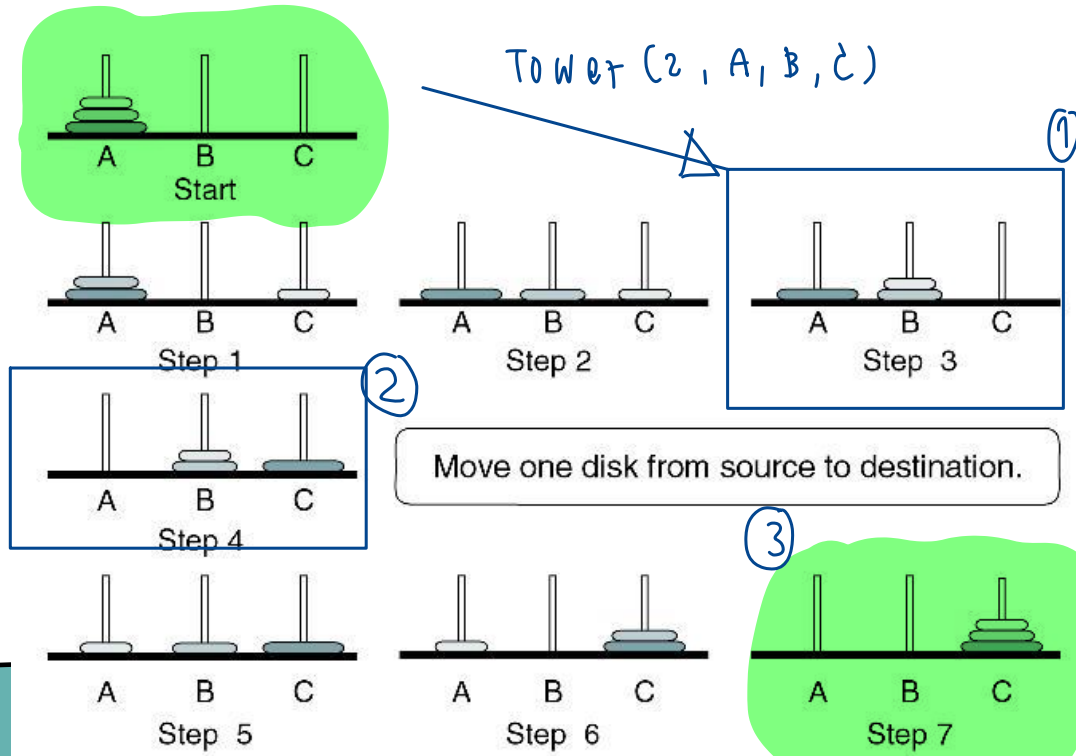
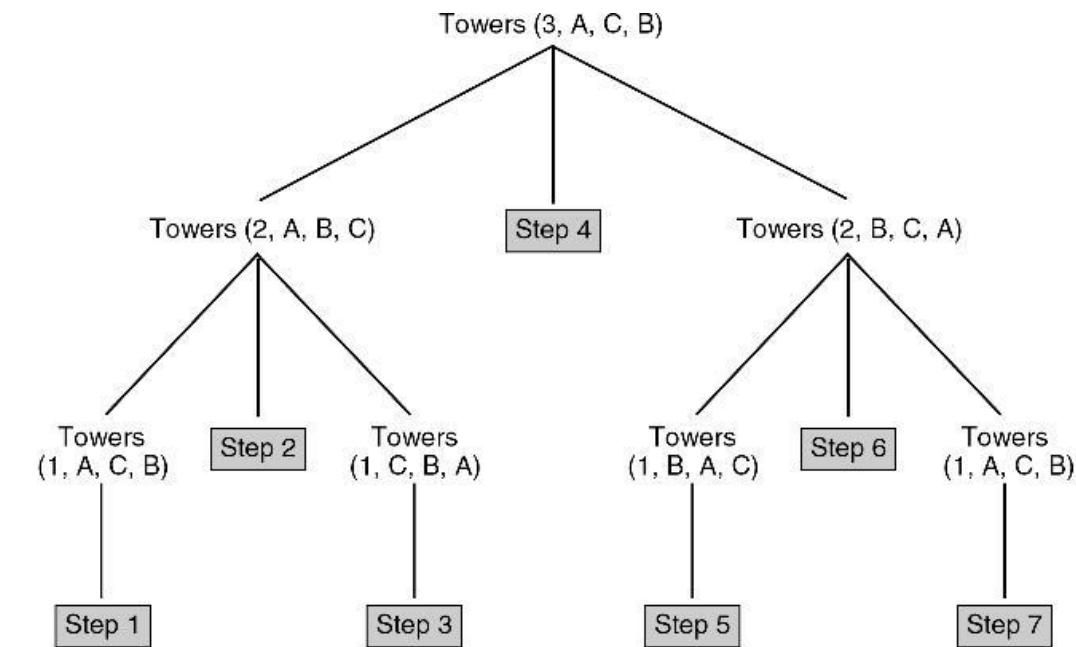
- ❖ ให้ทำการย้ายแผ่นดิสก์จากแกน Source ไปยังแกน Destination โดยที่
  - ดิสก์แผ่นเล็กต้องอยู่บนดิสก์แผ่นใหญ่เสมอ และสามารถย้ายแผ่นดิสก์ได้ครั้งละหนึ่งแผ่นเท่านั้น
  - ใช้แกน Auxiliary ช่วยพักแผ่นดิสก์ชั่วคราว (ช่วยย้ายแผ่นดิสก์ไปยังแกน Destination)



# TOWERS SOLUTION FOR TWO DISKS



# TOWERS SOLUTION FOR THREE DISKS





# TOWERS SOLUTION FOR <sup>n</sup>~~THREE~~ DISKS

1. Move  $n-1$  disks from source to auxiliary.
2. Move one disk from source to destination.
3. Move  $n-1$  disks from auxiliary to destination.

# THE TOWERS OF HANOI

```
Algorithm towers (numDisks, source, dest, auxiliary)
Recursively move disks from source to destination.
    Pre  numDisks is number of disks to be moved
        source, destination, and auxiliary towers given
    Post steps for moves printed
1  print("Towers: ", numDisks, source, dest, auxiliary)
2  if (numDisks is 1)
    1  print ("Move from ", source, " to ", dest)
3  else
    1  towers (numDisks - 1, source, auxiliary, dest, step)
    2  print ("Move from " source " to " dest)
    3  towers (numDisks - 1, auxiliary, dest, source, step)
4  end if
end towers
```

# THE TOWERS OF HANOI

## Calls:

Towers (3, A, C, B)

Towers (2, A, B, C)

Towers (1, A, C, B)

Towers (1, C, B, A)

Towers (2, B, C, A)

Towers (1, B, A, C)

Towers (1, A, C, B)

## Output:

Move from A to C

Move from A to B

Move from C to B

Move from A to C

Move from B to A

Move from B to C

Move from A to C

# EXERCISE 1

```
Algorithm 32ex_1(n)
    int r1
    if (n == 1)
        r1 = 12
    else
        r1 = n + ex_1(n/2)
    end if
    return (r1)
end ex_1
```

**ex\_1(32) = ?**

$$\begin{array}{l} \textcircled{74} \quad 32 + e(16) \\ \quad \downarrow \\ 42 = 16 + e(8) \\ \quad \downarrow \\ 26 = 8 + e(4) \\ \quad \downarrow \\ 18 = 4 + e(2) \\ \quad \downarrow \\ 14 = 2 + e(1) \\ \quad \downarrow \\ 12 \end{array}$$

# EXERCISE 2

```
Algorithm ex_2(0n)
  if (n==8)
    println("Go Back!")
  else
    println( n + " Hello!")
    ex_2(n+2)
    println( n + " Bye!")
  end if
end ex_2
```

```
0 Hello!
2 Hello!
4 Hello!
6 Hello!
Go Back!
6 Bye!
5 Bye!
4 Bye!
2 Bye!
0 Bye!
```

ex\_2(0) = ?

```
e(2)
↓
e(4)
↓
e(6)
↓
e(8)
```

# EXERCISE 3

Algorithm ex\_3(x)

```
    if (x<5)
        return (3*x)
    else
        return (2*ex_3(x-5)+7)
    end if
    println("Finish!!")
    return 1
end ex_3
```

$$(97) = 2 \cdot e(17) + 7$$

$$45 = 2 \cdot e(7) + 7$$

$$19 = 2 \cdot e(2) + 7$$

|              |                     |
|--------------|---------------------|
| ex_3(4) = ?  | 12<br>Finish!!<br>1 |
| ex_3(10) = ? | 21<br>Finish!!<br>1 |
| ex_3(17) = ? | 97<br>Finish!!<br>1 |